cim-mlir

An open, retargetable compiler and runtime stack for compute-in-memory and processing-in-memory hardware.

PROJECT STATUS REPORT

Architecture, delivered scope, and launch readiness — 17 August 2026

31,845	8	10	465	12
lines of code	compiler passes	dialect ops	automated tests	target files

Headline

A real quantized INT8 convolutional network — chained convolution layers, interleaved max-pooling, and a fully-connected head — now compiles from a standard **.onnx** file and executes, with every numerical claim checked against independent reference implementations rather than against this project's own code.

Milestones M0–M3 are complete and M4 is substantially complete. The remaining distance to a public v0.1 launch is dominated by one infrastructure blocker outside the codebase (exhausted CI minutes, resets 1 September 2026) and by the absence of measured hardware numbers — not by missing compiler capability.

Repository	Hajjy22/cim-mlir	**Branch**	claude/cim-mlir-cost-report-zhj2ce
HEAD	fe3aced	**Base**	main @ 8325ccb
Open PR	#22 (6 commits)	**Toolchain**	MLIR / LLVM 18

1 Executive summary

CIM/PIM hardware exists and some of it ships, but there is no open way to take a normal neural network and run it on that hardware without hand-writing assembly or using a vendor's closed, single-chip compiler. cim-mlir is a three-part open-source stack that closes that gap: an MLIR dialect and lowering pipeline, a thin C runtime ABI, and a benchmark suite with a cost model.

Where the project stands

The compiler is real and composes end to end. All eight lowering passes are implemented, and a single module has been driven through literally all eight in spec order, down to real runtime calls with no dialect ops left. The placement engine — the scheduling problem the project exists to solve — is implemented and proven optimal by exhaustive search on the shape the compiler actually emits.

The front end has moved from a single matrix multiply to a genuine convolutional network. Over the current development cycle it gained convolution-to-convolution chaining, a real MLIR-level im2col, a conv stem feeding a fully-connected head, and most recently the first executable non-matmul operator in the dialect — a windowed maximum — together with max-pooling interleaved into a convolution chain.

What is not yet true

No number in this project has been measured on real hardware. Every shipped target description is marked *estimated*, the hardware backend stubs every entry point, and one cost figure inherited from the specification is internally inconsistent by a factor of 1000 and must be resolved against a real device before it appears in a paper. These are disclosed in the repository rather than implied away, and they are the substance of what remains before launch.

Single critical blocker

GitHub Actions minutes for the account are exhausted for the billing cycle and reset on **1 September 2026**. All nine CI jobs fail instantly with empty output. There is no code-side fix. Development has continued against a full local verification gate that reproduces every CI job, and the work is staged on an open pull request ready to land the moment CI recovers.

2 Architecture

The stack is deliberately three separable pieces. The hardware is described by a portable YAML target file rather than hardcoded, so a vendor should be able to support cim-mlir by writing one file.

2.1 The three parts

Component	What it is	Key artifacts
Front end cim_frontend	Reads a standard ONNX model and emits the MLIR the pipeline consumes. Pure Python; no ONNX dependency below this layer. Refuses rather than guesses — dropping an unrecognised operator would emit a module that runs and computes a different function.	onnx_import.py, emit.py, im2col.py, analyze.py
Compiler cim dialect	An MLIR dialect of 10 operations plus 8 lowering passes that take standard ML graphs down to CIM primitives, then to real C ABI calls that go through MLIR's own LLVM pipeline into a linkable binary.	lib/Transforms/*, lib/Dialect/, lib/Placement/
Runtime cimrt	A thin C API of 20 entry points that executes the compiled artifact. Ships with a functional INT8 simulator; the real hardware backend is stubbed.	runtime/include/cimrt.h, runtime/src/simulator/
Benchmarks cim-bench	Reproducible workloads and an analytical cost model, so "is CIM actually better here?" is measurable rather than a marketing claim.	tools/cim-bench, bench/workloads/

2.2 The lowering pipeline

Eight passes, run in specification order. Passes 1–3 are the core of the project; pass 3 is the one it exists for.

#	Pass	Responsibility	State
1	`cim-detect`	Annotates INT8 matmuls that have exactly one constant weight operand; reports why it declined every other candidate.	Complete
2	`cim-partition`	Lowers a candidate into per-tile program/mvm pairs with partial-sum reduction and explicit memory-space transfers.	Complete
3	`cim-placement`	Rewrites that IR from a Belady schedule: eliminates redundant weight programming, assigns real tile ids, hoists loop-invariant programming out of loops when provably safe.	Complete
4	`cim-schedule`	Inserts barriers conservatively over placed IR. Does not reorder or overlap in v0.1.	Scoped
5	`cim-insert-transfers`	Inserts host-to-near copies where an activation is not already in near space, hoisting above a loop when the source is invariant.	Complete
6	`cim-legalize-precision`	Inserts a requantize after every terminal accumulator and warns when the target clamps below 8 bits.	Partial
7	`cim-lower-to-target`	Converts dialect ops into real C ABI calls so the result can become a linkable binary. Covers a deliberately scoped straight-line slice.	Scoped
8	`cim-cost-report`	Walks the final placed IR and emits the project's publishable energy and latency numbers.	Complete

2.3 The dialect

Ten operations. Nine were present from the first milestone; `cim.reduce_max` is new this cycle and is the first operation in the dialect that is not part of a matrix multiply.

Operation	Purpose	Interpreter	Compiled
`cim.device_open`	Open a target device	Yes	Yes
`cim.tile_alloc`	Reserve a physical tile	Yes	Yes
`cim.tile_free`	Release a tile	Yes	Yes
`cim.program`	Make a weight block resident (the expensive, asymmetric operation)	Yes	Yes
`cim.mvm`	Matrix multiply against a resident tile	Yes	Yes
`cim.reduce_partial`	Elementwise integer sum of N buffers (partial sums, and bias add)	Yes	Yes
`cim.reduce_max`	Elementwise SIGNED integer maximum of N buffers (a pooling window)	Yes	Missing
`cim.requantize`	Round, offset and clamp an accumulator	Yes	Yes
`cim.copy`	Explicit memory-space transfer	Yes	Yes
`cim.barrier`	Ordering	Yes	Yes

The single red cell is the most concrete open gap in the compiler and is addressed in section 5.

2.4 Runtime ABI and target description

The runtime is 20 C entry points. Every operation the runtime can execute is charged against the target's cost table — a project invariant, enforced by a differential test asserting that the statically predicted cost equals the cost actually measured at runtime. A new runtime call with no static counterpart is treated as a silent regression, not a missing feature.

```
cimrt_open, cimrt_close, cimrt_query, cimrt_alloc, cimrt_free, cimrt_map, cimrt_write, cimrt_read,
cimrt_copy, cimrt_copy_range, cimrt_program, cimrt_mvm, cimrt_requantize, cimrt_reduce_add,
cimrt_reduce_add_inplace, cimrt_reduce_max, cimrt_barrier, cimrt_profile_start, cimrt_profile_stop,
cimrt_status_string
```

Twelve target description files exist — three shipped (an Erbium-8T analog part, a generic digital CIM part, and a UPMEM-like near-memory part) and nine test fixtures that vary one capability each. The reader is hand-rolled, its supported YAML subset is written down rather than inherited, and it is checked differentially against PyYAML plus an independently written schema.

3 What has been built

3.1 Milestone status

Milestone	Scope	State
M0	Environment and orientation	Complete
M1	Dialect skeleton	Complete
M2	Functional correctness — a .onnx file compiles and runs, checked for exact integer equality against ONNX's own reference implementation	Complete
M3	The placement pass — Belady eviction, loop-invariant hoisting, and a steady-state pin-and-stream schedule proved optimal by exhaustive search	Complete
M4	Second target and generalization — most items closed, including full cost-model integrity, batching, convolution, and real-model analysis; calibration remains	Mostly
M5	Community and real hardware — hardware bring-up, upstream contribution, conference talk	Not started
M6	Decision point	Not started

3.2 Delivered this development cycle

Six commits on the open pull request, taking the front end from a single convolution to a genuine convolutional network.

Commit	Capability	Why it was hard
Conv → matmul	A convolution stem feeding a fully-connected head.	ONNX's own matmul kernel does N-D broadcast rather than requiring 2-D operands, so wiring a convolution's raw output straight in silently contracts the wrong axes instead of erroring. Fixed by requiring an explicit reshape bridge in the graph.
Interpreter reshape	Execute shape expansion and collapse.	Neither operation was executable at all. Needed before any MLIR-level im2col could reinterpret a buffer between matmul and gather shapes.
Conv → conv	Real convolution-to-convolution chaining, with a genuine MLIR-level im2col.	The interpreter cannot evaluate integer arithmetic or conditionals, ruling out both obvious im2col designs. Solved by unrolling over kernel taps as static strided views. The channel-last weight flatten for interior layers is a silent-wrong-answer hazard, mutation-tested accordingly.
Conv stem → head	The realistic full CNN shape.	Built by composing the two mechanisms above rather than reimplementing either.
reduce_max	The first executable non-matmul primitive.	Max on INT8 compares SIGNED, but the sibling addition is deliberately sign-agnostic — a copy-paste would compare raw bytes, compile, run, and quietly return the wrong answer. Landed with full cost accounting, which a first draft of the plan missed entirely.
MaxPool chaining	Pooling interleaved into a convolution chain.	The reference oracle cannot evaluate integer pooling at unit stride at all, so that case is refused on verifiability grounds and an independent hand-written oracle was added alongside.

3.3 Model coverage today

Seven distinct graph shapes are imported and compiled. Anything outside them is refused with a stated reason rather than silently approximated.

- A single INT8 matrix multiply, and chains of them.
- A single 2-D convolution, including per-channel scale, a real bias, asymmetric zero points, dilation, stride and padding.
- A convolution feeding one or more matmul layers.
- Chained convolutions, connected directly.
- A convolution stem feeding a fully-connected head.
- Chained convolutions with max-pooling interleaved between layers.
- Any graph at all, for placement and cost *analysis* without execution — a permissive walker classifies every node and never aborts, and discloses what it skipped.

4 Verification posture

The verification discipline is the project's main defence against its central failure mode: not a crash, but structurally valid code that computes a confidently wrong number.

Suite	Count	What it covers
ctest	20	End-to-end real compiled binaries: each numerically correct case paired with a deliberately wrong one that must trap.
cim-unit-tests	134	Runtime ABI, target parser, placement engine, cost model, workload reader.
cim-mlir-tests	43	In-process pipeline, precision legalization, and static-versus-runtime cost differentials.
pytest	268	The ONNX front end, driven against the real shipped binaries, plus differential fuzzing of the YAML reader against PyYAML.
lit / FileCheck	40 files	Dialect round-trip, verifier rejection cases, per-pass structure, and full-pipeline composition.

Practices applied throughout

- **Probe before code.** Every new IR shape is hand-verified through the real compiler and runtime binaries before any emission code is written to produce it.
- **Independent oracles.** Correctness is checked against implementations written by other people for other reasons — ONNX's reference implementation, onnxruntime, NumPy, PyYAML — not against this project's own second opinion.
- **Mutation testing.** Every new guard is deliberately broken, confirmed to fail for the right reason, then reverted and confirmed green. A test that has never been seen to fail is not treated as evidence.
- **Refuse rather than approximate.** Unsupported input is rejected with a stated reason. Several restrictions exist because the *oracle* cannot evaluate a case, not because the compiler cannot compile it — and say so explicitly.
- **Cost integrity as an invariant.** Statically predicted cost and runtime-measured cost are asserted equal by test.

A defect found in this project's own primary oracle

ONNX's reference implementation cannot evaluate an integer max-pool at unit stride at all: it pads with a floating-point sentinel that cannot be placed into an integer array, and raises rather than returning a wrong number. This was discovered by probing, recorded in the repository so it is not rediscovered, and is the stated reason that one configuration is refused — a gap in what can be verified, not in what can be compiled.

5 What is still open

5.1 Blockers

Item	Detail	Resolution
CI unavailable	All nine CI jobs fail instantly with empty output. GitHub Actions minutes are exhausted for the billing cycle. No code-side fix exists. Mitigated by a full local gate that reproduces every job, including sanitiser builds.	Resets automatically on 1 September 2026.

5.2 Capability gaps

Gap	Why it matters	Size
No compiled lowering for the pooling primitive	Every other dialect operation works on both the interpreter and the compiled path. A pooling network therefore runs under the interpreter but cannot be built into a real-target binary. A plan is written and ready to execute.	Small
No calibration	Requantization scales are fixed rather than derived per layer. The arithmetic is already general; what is missing is a calibration step to choose real values from data.	Medium
Activation and shape operators	Relu needs an asymmetric clamp the current requantize cannot express. Average-pooling needs a division primitive that exists nowhere in the stack. Concatenation and residual addition need a loader for branching, multi-producer graphs; every loader to date assumes a single linear chain.	Large
Control flow in the compiled path	The compiled lowering covers straight-line code and simple loops. Conditionals and loop-carried values are still refused.	Medium
Higher-rank strided slices in the compiled path	The compiled path moves one contiguous byte range; a genuinely strided gather has no runtime equivalent yet.	Medium

5.3 Credibility gaps — the ones that matter most for launch

Item	Detail
No measured hardware numbers	Every shipped target file is marked *estimated*. The runtime's hardware backend stubs every entry point and returns a no-device error. All published energy and latency figures are therefore modelled, not observed — which the repository states plainly, but which limits what can be claimed externally.
A 1000x inconsistency inherited from the specification	The specification's own worked example is internally inconsistent by three orders of magnitude on weight-programming energy. Which reading is correct decides whether installation cost is a rounding error or a material fraction of the per-inference budget — which is the crux of the project's central argument. It must be settled against real hardware before it appears in a paper.
Placeholder cost entries	The newest cost-table entry has no hardware measurement behind it on any modelled target. Each file marks it as a placeholder and gives it a deliberately distinct value so a miswiring shows up as a wrong number rather than a plausible one.

6 Launch readiness

Assessed against a public v0.1 release: the project is usable by someone who is not its author, its claims survive scrutiny, and there is a reason for a stranger to care.

Dimension	Assessment	State
Core capability	The compiler does what it claims. A real quantized CNN compiles and runs, and the placement engine — the actual intellectual contribution — is implemented and proved optimal on the shape it emits.	Ready
Correctness evidence	465 automated tests, independent oracles, mutation testing, and a static-versus-runtime cost invariant. Stronger than typical for a project at this stage.	Ready
Documentation	A hardware abstraction model, a dialect reference, a target-format specification, a front-end guide, and a single-page website. The roadmap records not just what was done but what was tried and rejected, and why.	Ready
Packaging	The front end installs as a Python package; the compiler builds with CMake against MLIR 18. A dedicated CI job guards packaging.	Ready
Retargetability	Three shipped target descriptions across three hardware classes, a documented YAML subset, and a differential test against an independent parser. A vendor can add a target by writing one file.	Ready
Continuous integration	Nine jobs covering sanitisers, valgrind, coverage, static analysis and packaging — all currently unavailable for reasons outside the codebase.	Blocked
Measured results	No number has been observed on real hardware, and one inherited figure is inconsistent by 1000x. This is the weakest dimension and the one most likely to be challenged.	Not ready
Operator coverage	Sufficient for a small classification network. Activations, average-pooling and branching topologies are absent, so most real-world models will still be refused.	Partial
External engagement	No upstream contribution, no talk submitted, no external users.	Not started

Assessment

The engineering is in better shape than the evidence base. The compiler, the runtime, the target abstraction and the verification discipline are all genuinely ready for other people to look at. What is not ready is the empirical claim: no figure has been observed on hardware, and the project's central argument about weight-programming amortisation rests on a specification figure that is internally inconsistent.

The most efficient next effort is therefore not more compiler features. It is landing the outstanding work, closing the one remaining dialect asymmetry, and then turning to measurement — because a smaller, honestly measured claim will survive scrutiny that a broader modelled one will not.

6.1 Critical path to launch

Step	Action	Depends on
1	Land the open pull request once CI recovers, confirming all nine jobs green on the four-commit convolution series and the two pooling commits.	CI reset, 1 Sep 2026
2	Close the compiled-path gap for the pooling primitive, so every dialect operation works on both execution paths. Plan written and approved.	Nothing
3	Resolve the 1000x energy inconsistency, or state a defensible reading and mark it clearly everywhere it is used.	Hardware access or a specification decision
4	Obtain at least one measured data point on real hardware, or reframe every published figure explicitly as modelled.	Hardware access
5	Decide the launch scope for operator coverage: ship as a convolutional-network compiler with stated limits, or first add activations and branching topologies.	A scope decision
6	External engagement — upstream contribution and a talk submission.	Steps 1 to 4

Steps 1 and 2 are entirely within the project's own control and account for the whole of the remaining engineering work. Steps 3 and 4 are the ones that decide how strong a claim the launch can make, and both depend on access to real hardware rather than on further development.